

JUnit & Static/Private Methods

1. How do you mock a static method in JUnit?

A: Standard JUnit and Mockito cannot mock static methods by themselves. You must use an external library like **PowerMock** or **Mockito with the mockito-inline dependency**. This allows you to define the mock behavior within a try-with-resources block using Mockito.mockStatic() for the duration of that specific test.

2. Should private methods be tested directly? How?

A: No, private methods should generally **not** be tested directly. They should only be tested indirectly through the public methods that use them. If you absolutely must test complex private logic, you can temporarily change its visibility to protected or package-private, or use Java Reflection to access it, but this is usually discouraged.

3. How do @RunWith and @Rule work in JUnit?

A: **@RunWith** (JUnit 4) tells JUnit to use a custom test runner (like SpringRunner or MockitoJUnitRunner) instead of the default one. **@Rule** (JUnit 4) allows you to define reusable setup/teardown logic (like setting up temporary folders or handling exceptions) that applies to every test method. JUnit 5 replaces these with @ExtendWith and @RegisterExtension.

4. How do you test methods that interact with the database without hitting the real DB?

A: You test these methods by replacing the real **Repository** (the class that talks to the DB) with a **mock object** using Mockito. You then tell the mock what data to return when the method is called. This lets you focus only on the business logic without the test relying on a slow, external database.

5. How does JUnit handle parallel test execution?

A: JUnit 5 handles parallel execution by using a specific configuration file to define a strategy (like executing tests by class or by method) and the thread pool size. This allows the testing framework to run multiple independent test methods or classes at the same time, speeding up the overall test suite.

Mockito: Advanced Mocking

6. How do you mock a void method?

A: You mock a void method using the **doNothing()** syntax to stop the method from executing its real code. If you want the void method to throw an error, you use **doThrow()**. These methods are placed before the when() call because void methods cannot return a value.

7. When should you use `doReturn()`, `doThrow()`, `doAnswer()`?

A: You use `doReturn()` when mocking a spy or when the method you are mocking has complex logic you want to bypass. You use `doThrow()` when you want the method to throw a specific error. You use `doAnswer()` when you need to run custom code that modifies the arguments or performs complex behavior when the mock method is called.

8. Can Mockito mock final classes/methods?

A: Yes, modern versions of Mockito (since 2.x) can mock final classes and methods without using external libraries like PowerMock. This feature is automatically enabled when you use the mockito-inline dependency and a recent JVM.

9. How do you mock dependencies passed as method parameters?

A: You use `ArgumentCaptor` or `Mockito.any(Class.class)` for objects passed as parameters. For very complex scenarios, you use `doAnswer()` to run custom logic that can access and modify the input parameter before returning a result.

10. How do you mock method chaining like `foo.bar().baz()`?

A: You mock method chaining by creating separate `when().thenReturn()` statements for each method call in the chain. You ensure the first `when()` call returns the mock object used in the second `when()` call, and so on. For simple stubs, you can sometimes use `RETURNS_DEEP_STUBS`.

11. How do you mock objects with constructor injection?

A: You mock constructor injection by using the `@InjectMocks` annotation on the class under test. Mockito uses reflection to find the correct constructor and automatically passes in all the mock objects defined with `@Mock`.

12. What is `RETURNS_DEEP_STUBS`, and when is it useful?

A: `RETURNS_DEEP_STUBS` is a setting that makes mock objects return **other mock objects** by default instead of returning null. It is useful for quickly stubbing long chains of methods (like `user.getAccount().getBalance()`) when you don't want to define the mock for every step in the chain.

13. How do you mock randomness like `Math.random()`?

A: You mock randomness by using an external library like **PowerMock** or by using `Mockito.mockStatic()` if your Mockito version supports it. This allows you to force the static `Math.random()` method to return a predictable, fixed value during your test execution.

TestNG: Configuration & Features

14. How do you run tests in parallel?

A: You run tests in parallel by configuring the **testng.xml** file. You set the parallel attribute (e.g., parallel="methods" or parallel="classes") and define the number of threads you want to use (thread-count).

15. dependsOnMethods usage.

A: **dependsOnMethods** is used to tell TestNG that one test method should only run if another specific test method has already run and passed successfully. This is useful for integration tests where one step (like creating a user) must succeed before the next step (like updating that user) can be attempted.

16. How do you retry failed tests in TestNG?

A: You retry failed tests by implementing the **IRetryAnalyzer** interface. You apply the custom retry class to your test method using the retryAnalyzer attribute in the @Test annotation. This tells TestNG to re-execute the test a defined number of times if it fails.

17. How do you generate HTML or Extent Reports?

A: You generate HTML or Extent Reports by using **listeners**. You implement a custom listener class that hooks into the TestNG execution process and writes the test status, logs, and results into a formatted HTML file as the tests run.

18. What are TestNG listeners?

A: **TestNG listeners** are objects that listen to the execution process and perform actions when specific events happen (e.g., a test starts, a test fails, a suite finishes). They are used for generating reports, logging, and sending notifications.

19. How do you implement ITestListener?

A: You implement **ITestListener** by creating a class that implements the interface and defining methods like onTestStart(), onTestFailure(), and onTestSuccess(). You then register this class in the **testng.xml** file so TestNG knows to call it.

20. Difference between testng.xml and @Test-level config.

A: **@Test** level configuration (using attributes like groups or priority) applies to a single method. **testng.xml** provides configuration that applies globally, such as setting up the entire test suite, defining parameters, and managing parallel execution settings.

21. invocationCount usage.

A: **invocationCount** is an attribute used in the `@Test` annotation to run the exact same test method multiple times sequentially. This is often used for simple performance testing or load testing scenarios.

22. Integrating TestNG with Maven.

A: You integrate TestNG with Maven by using the **maven-surefire-plugin** in your pom.xml. You configure the plugin to point to your **testng.xml** file, telling Maven to use TestNG instead of JUnit for running the tests during the test phase of the build lifecycle.

RestAssured: API Testing

23. How do you handle authentication (Basic/OAuth)?

A: You handle **Basic Authentication** using the `.auth().basic(username, password)` method in the `given()` section. You handle **OAuth2** (like a Bearer Token) using the `.auth().oauth2(token)` method or by simply adding the token via the header method: `.header("Authorization", "Bearer " + token)`.

24. How do you use RequestSpecification/ResponseSpecification?

A: **RequestSpecification** and **ResponseSpecification** are reusable objects that store common request details (like base URI, common headers, or authentication) and common response expectations (like `statusCode(200)`). You use them to avoid writing the same setup and validation code repeatedly across many different tests.

25. How do you validate JSON schema?

A: You validate the JSON schema by including a separate library like `json-schema-validator`. In the `then()` section, you use `.body(matchesJsonSchemaInClasspath("schema.json"))` to assert that the received JSON response structure matches the rules defined in your external schema file.

26. How do you test file upload/download?

A: You test **file uploads** using the `.multiPart("file", new File("path/to/file"))` method in the `given()` section. You test **file downloads** by extracting the response as a byte array or stream and saving it to a temporary location for validation.

27. How do you test cookies?

A: You test cookies in the `given()` section by adding cookies to the request using `.cookie("session_id", "value")`. In the `then()` section, you assert the presence and value of cookies in the response using `.cookie("name", "expectedValue")`.

28. How do you test form-data/multipart?

A: You test form-data (HTML form submissions) using the `.formParam("key", "value")` method. You test **multipart** data (usually for file uploads) using the `.multiPart("controlName", "value")` or `.multiPart("file", fileObject)` methods in the `given()` section.

29. How do you capture response time?

A: You capture response time by using the `.time()` method in the `then()` section to assert that the entire request took less than a specified duration (e.g., `.time(lessThan(1000L))`). You can also use `.extract().time()` to pull the time into a variable.

30. How do you chain multiple API calls using extracted values?

A: You chain API calls by using the `.extract()` method after the first request. This pulls a necessary value (like an id or a token) from the JSON response body. You then use that extracted value as a parameter or header in the `given()` section of the next API call.